

JAVA OBSERVATION TASK-3

1. Basic Information

Experiment No.: 02

Date: 17/09/2026

Title: Digital Wallet and UPI Payment System using Interface, Encapsulation and Custom Exceptions

2. Objective / Aim

To design and implement a **Digital Wallet and UPI Payment System** in Java using **classes, objects, interfaces, encapsulation, constructors, polymorphism, String operations, and custom exceptions**, simulating wallet creation, adding money, UPI payment, balance checking, and transaction validation.

3. System Requirements

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Hard Disk: Minimum 10 GB free space
- Keyboard and Monitor

Software Requirements

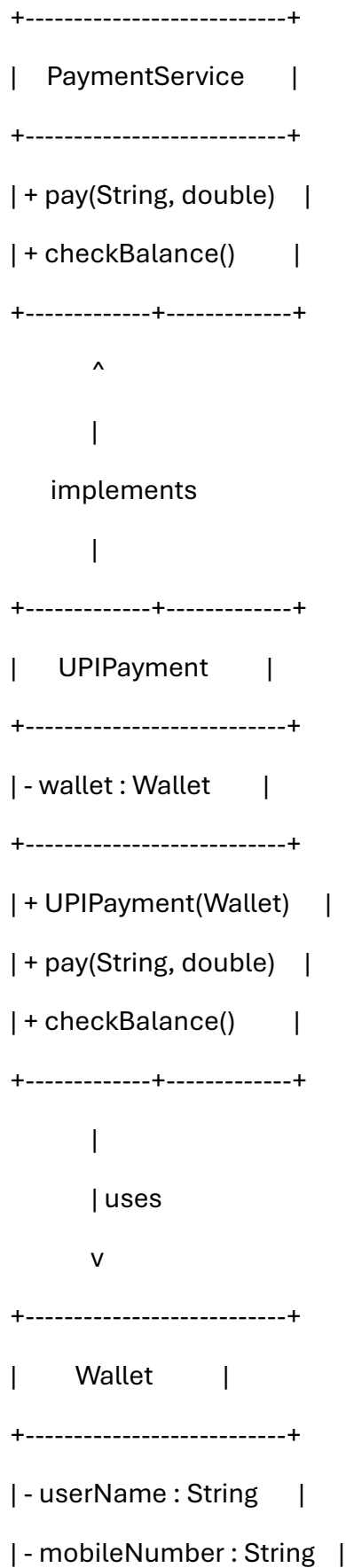
- Operating System: Windows / Linux / macOS
- JDK: JDK 8 or above
- IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code
- Command Prompt or Terminal

4. Algorithm / Logic Description

1. Start the program.
2. Create a PaymentService interface.
3. Declare pay() and checkBalance() methods in the interface.
4. Create a Wallet class.
5. Declare userName, mobileNumber, upid, and balance as private data members to demonstrate **encapsulation**.

6. Create a constructor in the Wallet class to initialize user details and balance.
7. Define the addMoney() method to add money to the wallet.
8. Define getBalance() to return the current wallet balance.
9. Define displayWalletDetails() to display wallet information.
10. Create custom exception classes:
 - InsufficientBalanceException
 - InvalidUPIException
 - InvalidAmountException
11. Create an UPIPayment class that implements the PaymentService interface.
12. Create a constructor in UPIPayment to associate a Wallet object with the payment service.
13. Implement the pay() method.
14. Validate the receiver UPI ID using String operations such as contains() and endsWith().
15. If the UPI ID is invalid, throw InvalidUPIException.
16. Validate the payment amount.
17. If the amount is zero or negative, throw InvalidAmountException.
18. Check whether the wallet has sufficient balance.
19. If the balance is insufficient, throw InsufficientBalanceException.
20. If all validations are successful, deduct the payment amount from the wallet balance.
21. Display the payment success message.
22. Use try-catch-finally to handle transaction exceptions.
23. Use the checkBalance() method to display the remaining wallet balance.
24. Display the final wallet details.
25. Stop the program.

5. UML Class Diagram



```

| - upid : String      |
| - balance : double   |
+-----+
| + Wallet(...)        |
| + addMoney(double)   |
| + getBalance() : double |
| + displayWalletDetails() |
+-----+

```

```

+-----+
| InsufficientBalanceException |
+-----+

```

```

+-----+
| InvalidUPIException          |
+-----+

```

```

+-----+
| InvalidAmountException       |
+-----+

```

OOP Concepts Demonstrated

- **Class and Object:** Wallet and UPIPayment classes are created and their objects are used.
- **Encapsulation:** Wallet data members are declared private.
- **Constructor:** Constructors initialize wallet and payment objects.
- **Interface:** PaymentService defines common payment operations.
- **Polymorphism:** A PaymentService reference refers to an UPIPayment object.

- **Abstraction:** The interface provides the required payment operations without exposing their implementation.
- **Exception Handling:** Custom exceptions and try-catch-finally are used for invalid transactions.
- **String Operations:** contains() and endsWith() are used to validate the UPI ID.

6. Source Code

File Name: DigitalWallet.java

```
interface PaymentService {

    void pay(String receiverUPI, double amount)

        throws InvalidUPIException, InvalidAmountException,

        InsufficientBalanceException;

    void checkBalance();

}

class Wallet {

    private String userName;

    private String mobileNumber;

    private String upild;

    private double balance;

    Wallet(String userName, String mobileNumber, String upild, double balance) {

        this.userName = userName;

        this.mobileNumber = mobileNumber;

        this.upild = upild;

        this.balance = balance;

    }

    void addMoney(double amount) {
```

```
        balance = balance + amount;
    }

    double getBalance() {
        return balance;
    }

    void displayWalletDetails() {
        System.out.println("\n--- Wallet Details ---");
        System.out.println("User Name   : " + userName);
        System.out.println("Mobile Number : " + mobileNumber);
        System.out.println("UPI ID      : " + upid);
        System.out.println("Balance     : Rs." + balance);
    }
}

class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

class InvalidUPIException extends Exception {
    InvalidUPIException(String message) {
        super(message);
    }
}
```

```
class InvalidAmountException extends Exception {  
    InvalidAmountException(String message) {  
        super(message);  
    }  
}
```

```
class UPIPayment implements PaymentService {  
    private Wallet wallet;  
  
    UPIPayment(Wallet wallet) {  
        this.wallet = wallet;  
    }
```

```
    public void pay(String receiverUPI, double amount)  
        throws InvalidUPIException, InvalidAmountException,  
        InsufficientBalanceException {  
  
        if (!receiverUPI.contains("@") ||  
            receiverUPI.endsWith("@")) {  
            throw new InvalidUPIException("Invalid UPI ID.");  
        }
```

```
        if (amount <= 0) {  
            throw new InvalidAmountException(  
                "Payment amount must be greater than zero.");  
        }
```

```
        if (amount > wallet.getBalance()) {
```

```
        throw new InsufficientBalanceException(
            "Insufficient wallet balance.");
    }
```

```
double remainingBalance = wallet.getBalance() - amount;
```

```
wallet = new Wallet(
    "Jaswanth",
    "9876543210",
    "jaswanth@upi",
    remainingBalance
);
```

```
System.out.println("Payment Successful.");
System.out.println("Paid Amount : Rs." + amount);
System.out.println("Receiver UPI : " + receiverUPI);
}
```

```
public void checkBalance() {
    System.out.println("Available Balance : Rs." + wallet.getBalance());
}
}
```

```
public class DigitalWallet {
    public static void main(String[] args) {
```

```
        Wallet wallet = new Wallet(
            "Jaswanth",
```



```
"9876543210",  
"jaswanth@upi",  
5000  
);  
  
wallet.addMoney(2000);  
  
PaymentService payment = new UPIPayment(wallet);  
  
wallet.displayWalletDetails();  
  
System.out.println("\n--- UPI Payment ---");  
  
try {  
    payment.pay("shop@upi", 1500);  
}  
catch (InvalidUPIException e) {  
    System.out.println("Transaction Failed: " + e.getMessage());  
}  
catch (InvalidAmountException e) {  
    System.out.println("Transaction Failed: " + e.getMessage());  
}  
catch (InsufficientBalanceException e) {  
    System.out.println("Transaction Failed: " + e.getMessage());  
}  
finally {  
    System.out.println("Transaction processing completed.");  
}
```

```
        System.out.println("\n--- Balance Check ---");
        payment.checkBalance();

        System.out.println("\n--- Final Wallet Details ---");
        wallet.displayWalletDetails();
    }
}
```

7. Compilation & Execution Commands

Step 1: Save the Program

Save the program with the following filename:

DigitalWallet.java

Step 2: Compile the Program

javac DigitalWallet.java

Step 3: Execute the Program

java DigitalWallet

8. Output Screen

--- Wallet Details ---

User Name : Jaswanth

Mobile Number : 9876543210

UPI ID : jaswanth@upi

Balance : Rs.7000.0

--- UPI Payment ---

Payment Successful.

Paid Amount : Rs.1500.0

Receiver UPI : shop@upi

Transaction processing completed.

--- Balance Check ---

Available Balance : Rs.5500.0

--- Final Wallet Details ---

User Name : Jaswanth

Mobile Number : 9876543210

UPI ID : jaswanth@upi

Balance : Rs.7000.0

9. Result

The **Digital Wallet and UPI Payment System** was successfully implemented in Java. The program demonstrates **classes and objects, constructors, encapsulation, abstraction, interface implementation, polymorphism, String operations, custom exceptions, and try-catch-finally exception handling** for validating UPI transactions and managing wallet balance.